CS-462 – Programming Language Foundations
Lecture #11: deBruijn Notation and Definitional Translation

EPFL

1 de Bruijn Notation

One way to avoid the tricky interaction between free and bound names in the substitution operator is to pick a representation for expressions that doesn't have any names at all! Intuitively, we can think of a bound variable is just a pointer to the λ that binds it. For example, in $\lambda x.\lambda y.y\ x$, the y points to the first λ and the x points to the second λ.

So-called *de Bruijn* notation uses this idea as the representation for λ expressions. Here is the grammar for λ expressions in de Bruijn notation:

$$e ::= n \mid \lambda.e \mid e\ e$$

Variables are represented by integers n that refer to (the index of) their binder while *lambda*-abstractions have the form $\lambda.e$. Note that the the variable bound by the abstraction is not named—i.e., the representation is nameless.

As examples, here are several terms written using standard notation and in de Bruijn notation:

Standard	de Bruijn
$\lambda x.x$	$\lambda.0$
$\lambda z.z$	$\lambda.0$
$\lambda x.\lambda y.x$	$\lambda.\lambda.1$
$\lambda x.\lambda y.\lambda s.\lambda z.x\ s\ (y\ s\ z)$	$\lambda.\lambda.\lambda.\lambda.3\ 1\ (2\ 1\ 0)$
$(\lambda x.x\ x)\ (\lambda x.x\ x)$	$(\lambda.0\ 0)\ (\lambda.0\ 0)$
$(\lambda x.\lambda x.x)\ (\lambda y.y)$	$(\lambda.\lambda.0)\ (\lambda.0)$

To represent a λ-expression that contains free variables in de Bruijn notation, we need a way to map the free variables to integers. We will work with respect to a map Γ from variables to integers called a *context*. As an example, if Γ maps x to 0 and y to 1, then the de Bruijn representation of $x\ y$ with respect to Γ is $0\ 1$, while the representation of $\lambda z.\ x\ y\ z$ with respect to Γ is $\lambda.\ 1\ 2\ 0$. Note that in this second example, because we have gone under a λ, we have shifted the integers representing x and y up by one to avoid capturing them.

In general, whenever we work de Bruijn representations of expressions containing free variables (i.e., when working with respect to a context Γ) we will need to modify the indices of those variables. For example, when we substitute an expression containing free variables under a λ, we will need to shift the indices up so that they continue to refer to the same numbers with respect to Γ after the substitution as they did before. For example, if we substitute $0\ 1$ for the variable bound by the outermost λ in $\lambda.\lambda.1$ we should get $\lambda.\lambda.2\ 3$, not $\lambda.\lambda.0\ 1$. We will use an auxiliary function

that shifts the indices of free variables above a cutoff c up by i:

$$\begin{array}{rcl} \uparrow_c^i (n) & = & \begin{cases} n & \text{if } n < c \\ n + i & \text{otherwise} \end{cases} \\ \uparrow_c^i (\lambda.e) & = & \lambda.(\uparrow_{c+1}^i e) \\ \uparrow_c^i (e_1\ e_2) & = & (\uparrow_c^i\ e_1)\ (\uparrow_c^i\ e_2) \end{array}$$

The cutoff keeps track of the variables that were bound in the original expression and so should not be shifted as the shifting operator walks down the structure of an expression. The cutoff is 0 initially.

Using this shifting function, we can define substitution as follows:

$$\begin{array}{rcl} n\{e/m\} & = & \begin{cases} e & \text{if } n = m \\ n & \text{otherwise} \end{cases} \\ (\lambda.e_1)\{e/m\} & = & \lambda.e_1\{(\uparrow_0^1 e)/m+1\})) \\ (e_1\ e_2)\{e/m\} & = & (e_1\{e/m\})\ (e_2\{e/m\}) \end{array}$$

Note that when we go under a λ we increase the index of the variable we are substituting for and shift the free variables in the expression e up by one.

The β rule for terms in de Bruijn notation is as follows:

$$\beta\ \frac{}{(\lambda.e_1)\ e_2 \rightarrow \uparrow_0^{-1} (e_1\{\uparrow_0^1 e_2/0\})}$$

That is, we substitute occurrences of 0, the index of the variable being bound by the λ, with e_2 shifted up by one. Then we shift the result down by one to ensure that any free variables in e_1 continue to refer to the same things after we remove the λ.

To illustrate how this works consider the following example, which we wrote as $(\lambda u.\lambda v.u\ x)\ y$ in standard notation. We will work with respect to a context where $\Gamma(x) = 0$ and $\Gamma(y) = 1$.

$$\begin{array}{rl} & (\lambda.\lambda.1\ 2)\ 1 \\ \rightarrow & \uparrow_0^{-1} ((\lambda.1\ 2)\{(\uparrow_0^1 1)/0\}) \\ = & \uparrow_0^{-1} ((\lambda.1\ 2)\{2/0\}) \\ = & \uparrow_0^{-1} \lambda.((1\ 2)\{(\uparrow_0^1 2)/(0+1)\}) \\ = & \uparrow_0^{-1} \lambda.((1\ 2)\{3/1\}) \\ = & \uparrow_0^{-1} \lambda.(1\{3/1\})\ (2\{3/1\}) \\ = & \uparrow_0^{-1} \lambda.3\ 2 \\ = & \lambda.2\ 1 \end{array}$$

which, in standard notation (with respect to Γ), is the same as $\lambda v.y\ x$.

2 Definitional translation

We have seen how to encode a number of high-level language constructs—booleans, conditionals, natural numbers, and recursion—in λ-calculus. We now consider definitional translation, where we define the meaning of language constructs by translation to another language. This is a form of denotational semantics, but instead of the target being mathematical objects, it is a simpler programming language (such as λ-calculus). Note that definitional translation does not necessarily

produce clean or efficient code; rather, it defines the meaning of the source language constructs in terms of the target language.

For each language construct, we will define an operational semantics directly, and then give an alternate semantics by translation to a simpler language. We will start by introducing *evaluation contexts*, which make it easier to present the new language features succinctly.

2.1 Evaluation contexts

Recall the syntax and CBV operational semantics for the lambda calculus:

$$e ::= x \mid \lambda x.\, e \mid e_1\ e_2$$
$$v ::= \lambda x.\, e$$

$$\frac{e_1 \to e_1'}{e_1\ e_2 \to e_1'\ e_2} \qquad \frac{e_2 \to e_2'}{v_1\ e_2 \to v_1\ e_2'} \qquad \beta\text{-reduction}\ \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}$$

Of the operational semantics rules, only the β-reduction rule told us how to "reduce" an expression; the other two rules tell us the order to evaluate expressions—e.g., evaluate the left hand side of an application to a value first, then evaluate the right hand side of an application to a value. The operational semantics of many of the languages we will consider have this feature: there are two kinds of rules, *congruence rules* that specify evaluation order, and the *computation rules* that specify the "interesting" reductions.

Evaluation contexts are a simple mechanism that separates out these two kinds of rules. An evaluation context E (sometimes written $E[\cdot]$) is an expression with a "hole" in it, that is with a single occurrence of the special symbol $[\cdot]$ (called the "hole") in place of a subexpression. Evaluation contexts are defined using a BNF grammar that is similar to the grammar used to define the language. The following grammar defines evaluation contexts for the pure CBV λ-calculus.

$$E ::= [\cdot] \mid E\ e \mid v\ E$$

We write $E[e]$ to mean the evaluation context E where the hole has been replaced with the expression e. The following are examples of evaluation contexts, and evaluation contexts with the hole filled in by an expression.

$$E_1 = [\cdot]\ (\lambda x.\, x) \qquad E_1[\lambda y.\, y\ y] = (\lambda y.\, y\ y)\ \lambda x.\, x$$
$$E_2 = (\lambda z.\, z\ z)\ [\cdot] \qquad E_2[\lambda x.\, \lambda y.\, x] = (\lambda z.\, z\ z)\ (\lambda x.\, \lambda y.\, x)$$
$$E_3 = ([\cdot]\ \lambda x.\, x\ x)\ ((\lambda y.\, y)\ (\lambda y.\, y)) \qquad E_3[\lambda f.\, \lambda g.\, f\ g] = ((\lambda f.\, \lambda g.\, f\ g)\ \lambda x.\, x\ x)\ ((\lambda y.\, y)\ (\lambda y.\, y))$$

Using evaluation contexts, we can define the evaluation semantics for the pure CBV λ-calculus with just two rules, one for evaluation contexts, and one for β-reduction.

$$\frac{e \to e'}{E[e] \to E[e']} \qquad \beta\text{-reduction}\ \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}$$

Note that the evaluation contexts for the CBV λ-calculus ensure that we evaluate the left hand side of an application to a value, and then evaluate the right hand side of an application to a value before applying β-reduction.

We can specify the operational semantics of CBN λ-calculus using evaluation contexts:

$$E ::= [\cdot] \mid E\ e \qquad \frac{e \to e'}{E[e] \to E[e']} \qquad \beta\text{-REDUCTION}\ \frac{}{(\lambda x.\, e_1)\ e_2 \to e_1\{e_2/x\}}$$

We'll see the benefit of evaluation contexts as we see languages with more syntactic constructs.

2.2 Multi-argument functions and currying

Our syntax for functions only allows function with a single argument: $\lambda x.\, e$. We could define a language that allows functions to have multiple arguments.

$$e ::= x \mid \lambda x_1, \ldots, x_n.\, e \mid e_0\ e_1\ \ldots\ e_n$$

Here, a function $\lambda x_1, \ldots, x_n.\, e$ takes n arguments, with names x_1 through x_n. In a multi argument application $e_0\ e_1\ \ldots\ e_n$, we expect e_0 to evaluate to an n-argument function, and $e_1, \ldots, e_n$ are the arguments that we will give the function.

We can define a CBV operational semantics for the multi-argument λ-calculus as follows.

$$E ::= [\cdot] \mid v_0\ \ldots\ v_{i-1}\ E\ e_{i+1}\ \ldots\ e_n \qquad \frac{e \to e'}{E[e] \to E[e']}$$

$$\beta\text{-REDUCTION}\ \frac{}{(\lambda x_1, \ldots, x_n.\, e_0)\ v_1\ \ldots\ v_n \to e_0\{v_1/x_1\}\{v_2/x_2\}\ldots\{v_n/x_n\}}$$

The evaluation contexts ensure that we evaluate a multi-argument application $e_0\ e_1\ \ldots\ e_n$ by evaluating each expression from left to right down to a value.

Now, the multi-argument λ-calculus isn't any more expressive that the pure λ-calculus. We can show this by showing how any multi-argument λ-calculus program can be translated into an equivalent pure λ-calculus program. We define a translation function $\mathcal{T}[\![\cdot]\!]$ that takes an expression in the multi-argument λ-calculus and returns an equivalent expression in the pure λ-calculus. That is, if e is a multi-argument lambda calculus expression, $\mathcal{T}[\![e]\!]$ is a pure λ-calculus expression.

We define the translation as follows.

$$\begin{aligned}
\mathcal{T}[\![x]\!] &= x \\
\mathcal{T}[\![\lambda x_1, \ldots, x_n.\, e]\!] &= \lambda x_1.\ \ldots\ \lambda x_n.\, \mathcal{T}[\![e]\!] \\
\mathcal{T}[\![e_0\ e_1\ e_2\ \ldots\ e_n]\!] &= (\ldots((\mathcal{T}[\![e_0]\!]\ \mathcal{T}[\![e_1]\!])\ \mathcal{T}[\![e_2]\!])\ldots\ \mathcal{T}[\![e_n]\!])
\end{aligned}$$

This process of rewriting a function that takes multiple arguments as a chain of functions that each take a single argument is called *currying*. Consider a mathematical function that takes two arguments, the first from domain A and the second from domain B, and returns a result from domain C. We could describe this function, using mathematical notation for domains of functions, as being an element of $A \times B \to C$. Currying this function produces a function that is an element of $A \to (B \to C)$. That is, the curried version of the function takes an argument from domain A, and returns a function that takes an argument from domain B and produces a result of domain C.

3 Products and let

A product is a pair of expressions (e_1, e_2). If e_1 and e_2 are both values, then we regard the product as also being a value. (That is, we cannot further evaluate a product if both elements are values.) Given a product, we can access the first or second element using the operators $\#1$ and $\#2$ respectively. That is, $\#1\ (v_1, v_2) \to v_1$ and $\#2\ (v_1, v_2) \to v_2$. (Other common notation for projection includes π_1 and π_2, and fst and snd.)

The syntax of λ-calculus extended with products and let expressions is defined as follows.

$$\begin{aligned} e &::= x \mid \lambda x.\, e \mid e_1\ e_2 \\ &\mid (e_1, e_2) \mid \#1\ e \mid \#2\ e \\ &\mid \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \\ v &::= \lambda x.\, e \mid (v_1, v_2) \end{aligned}$$

Note that values in this language are either functions or pairs of values.

We define a small-step CBV operational semantics for the language using evaluation contexts.

$$E ::= [\cdot] \mid E\ e \mid v\ E \mid (E, e) \mid (v, E) \mid \#1\ E \mid \#2\ E \mid \mathsf{let}\ x = E\ \mathsf{in}\ e_2$$

$$\frac{e \to e'}{E[e] \to E[e']} \qquad \beta\text{-REDUCTION}\ \frac{}{(\lambda x.\, e)\ v \to e\{v/x\}}$$

$$\frac{}{\#1\ (v_1, v_2) \to v_1} \qquad \frac{}{\#2\ (v_1, v_2) \to v_2}$$

$$\frac{}{\mathsf{let}\ x = v\ \mathsf{in}\ e \to e\{v/x\}}$$

Next, we define an equivalent semantics by translation to the pure CBV λ-calculus.

$$\begin{aligned} \mathcal{T}\ [\![x]\!] &= x \\ \mathcal{T}\ [\![\lambda x.\, e]\!] &= \lambda x.\, \mathcal{T}\ [\![e]\!] \\ \mathcal{T}\ [\![e_1\ e_2]\!] &= \mathcal{T}\ [\![e_1]\!]\ \mathcal{T}\ [\![e_2]\!] \\ \mathcal{T}\ [\![(e_1, e_2)]\!] &= (\lambda x.\, \lambda y.\, \lambda f.\, f\ x\ y)\ \mathcal{T}\ [\![e_1]\!]\ \mathcal{T}\ [\![e_2]\!] \\ \mathcal{T}\ [\![\#1\ e]\!] &= \mathcal{T}\ [\![e]\!]\ (\lambda x.\, \lambda y.\, x) \\ \mathcal{T}\ [\![\#2\ e]\!] &= \mathcal{T}\ [\![e]\!]\ (\lambda x.\, \lambda y.\, y) \\ \mathcal{T}\ [\![\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2]\!] &= (\lambda x.\, \mathcal{T}\ [\![e_2]\!])\ \mathcal{T}\ [\![e_1]\!] \end{aligned}$$

Note that we encode a pair (e_1, e_2) as a value that takes a function f, and applies f to v_1 and v_2, where v_1 and v_2 are the result of evaluating e_1 and e_2 respectively. The projection operators pass a function to the encoding of pairs that selects either the first or second element as appropriate. Also note that the expression $\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2$ is equivalent to the application $(\lambda x.\, e_2)\ e_1$.

4 Laziness

In previous lectures we defined semantics for both the call-by-name λ-calculus and the call-by-value λ-calculus. It turns out that we can translate a call-by-name program into a call-by-value program. In CBV, arguments to functions are evaluated before the function is applied; in CBN, functions are applied as soon as possible. In the translation, we delay the evaluation of arguments by wrapping them in a function. This is called a *thunk*: wrapping a computation in a function to delay its evaluation.

Since arguments to functions are turned into thunks, when we want to use an argument in a function body, we need to evaluate the thunk. We do so by applying the thunk (which is simply a function); it doesn't matter what we apply the thunk to, since the thunk's argument is never used.

$$\begin{aligned}
\mathcal{T}\,[\![x]\!] &= x\ (\lambda y.\ y) \\
\mathcal{T}\,[\![\lambda x.\ e]\!] &= \lambda x.\ \mathcal{T}\,[\![e]\!] \\
\mathcal{T}\,[\![e_1\ e_2]\!] &= \mathcal{T}\,[\![e_1]\!]\ (\lambda z.\ \mathcal{T}\,[\![e_2]\!]) \qquad\qquad z \text{ is not a free variable of } e_2
\end{aligned}$$

5 Adequacy of translation

In each of the previous translations, we defined a semantics for the source language (using evaluation contexts and small-step rules) and the target language (by translation). We would like to be able to show that the translation is correct—that is, that it preserves the meaning of source programs.

More precisely, we would like an expression e in the source language to evaluate to a value v if and only if the translation of e evaluates to a value v' such that v' is "equivalent to" v. What exactly it means for v' to be "equivalent to" v will depend on the translation. Sometimes, it will mean that v' is literally the translation of v; other times, it will mean that v' is merely related to the translation of v by some equivalence.

One tricky issue is that in general, there can be many ways to define equivalences on functions. One way is to say that two functions are equivalent if they agree on the result when applied to any value of a base type (e.g., integers or booleans). The idea is that if two functions disagree when passed a more complex value (say, a function), then we could write a program that uses these functions to produce functions that disagree on values of base types.

There are two criteria for a translation to be *adequate*: soundness and completeness. For clarity, let's suppose that $\mathbf{Exp}_{\text{src}}$ is the set of source language expressions, and that $\rightarrow_{\text{src}}$ and $\rightarrow_{\text{trg}}$ are the evaluation relations for the source and target languages respectively. A translation is sound if every target evaluation represents a source evaluation:

$$\text{Soundness: } \forall e \in \mathbf{Exp}_{\text{src}}.\ \text{if } \mathcal{T}\,[\![e]\!] \rightarrow^*_{\text{trg}} v' \text{ then } \exists v.\ e \rightarrow^*_{\text{src}} v \text{ and } v' \text{ equivalent to } v$$

A translation is complete if every source evaluation has a target evaluation.

$$\text{Completeness: } \forall e \in \mathbf{Exp}_{\text{src}}.\ \text{if } e \rightarrow^*_{\text{src}} v \text{ then } \exists v'.\ \mathcal{T}\,[\![e]\!] \rightarrow^*_{\text{trg}} v' \text{ and } v' \text{ equivalent to } v$$